sft-label

SFT 训练数据的自动化质量评估流水线

Label · Score · Aggregate · Cluster · Filter

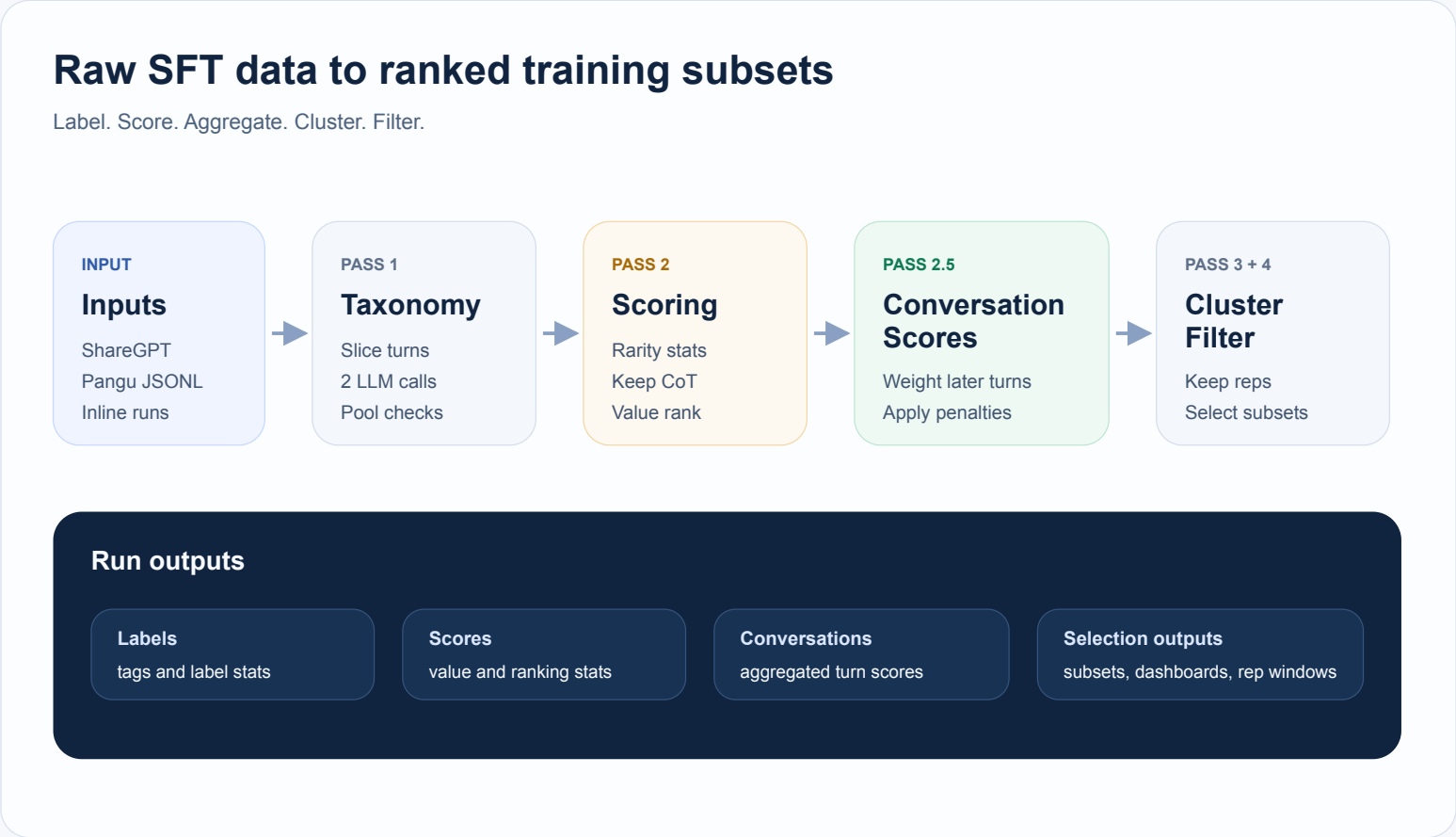
当前 SFT 数据 Curation 的痛点

- 数据量大，人工逐条审核不现实，标准难统一
- 多轮对话需要 **逐 turn** 评估，整体看会漏掉局部质量问题
- 单一总分筛选丢失信号 — 不知道 **为什么** 好或差
- 高质量但重复的数据挤占训练配额，缺少多样性控制

sft-label 的思路：**LLM-as-Judge 多维评分 + 对话级聚合 + 多样性感知筛选。**

端到端流水线总览

从原始 SFT 对话数据出发，经过标注、评分、聚合、聚类、筛选，输出高质量训练子集。



Pass 1 — 9 维分类标注，225+ 标签

每个样本经过 **2 次 LLM 调用**，标注 9 个维度：

维度	类型	标签数	说明
Intent	单选	6	用户的核心目标：learn / build / debug / review / modify / decide
Language	多选	75	对话中涉及的编程语言和配置格式
Domain	多选	38	应用领域：web-backend / ML / devops / cybersecurity…
Task	多选	22	工作类型：feature-impl / bug-fixing / refactoring…
Difficulty	单选	5	产出该回答所需的能力等级：beginner → expert
Concept	层级多选	26	编程知识：algorithms / concurrency / design-patterns…
Agentic	层级多选	23	Agent 能力：file-ops / shell-exec / multi-file-coord…
Constraint	多选	20	非功能约束：fault-tolerant / thread-safe / scalable…
Context	单选	10	代码范围：snippet → repository / greenfield / legacy…

Pass 1 — 两阶段 LLM 调用

第 1 次调用 → 5 个基础维度

Intent · Language · Domain · Task · Difficulty

这些维度相对独立，可从对话内容直接判断。

第 2 次调用 → 4 个上下文相关维度

Concept · Agentic · Constraint · Context

依赖第 1 次结果。例如知道 Domain=ML + Task=optimization，才能准确判断 Concept 是否含 `algorithms` 。

标注原则（Prompt 节选）

```
1. Evidence-based only
   每个标签必须有对话文本中的
   直接证据，不得推测

2. Conservative
   不确定时宁可不标
   漏标比错标危害更小

3. Label the last turn
   多轮对话以最后一轮 user query
   及其 response 为标注对象
   前文仅作上下文

4. Label required capability
   标注"产出该回答所需的能力"
   而非"对话讨论了什么"
```

Pass 1 — Prompt 节选：Difficulty 锚点

Prompt 中为每个维度提供了详细的锚点定义，以 Difficulty 为例：

```
### Difficulty (single-select)
What coding ability level is needed to PRODUCE a good response?

- beginner:    Basic syntax/stdlib, simple API calls, one-step fixes
- intermediate:
    Routine framework work with known patterns in local scope
    (Flask CRUD, React state form, straightforward SQL joins)
- upper-intermediate:
    Non-trivial but standard engineering coordination
    (2-5 files/modules, async refactor with error-path preservation, cross-component debugging)
- advanced:
    Hard system design/optimization with explicit tradeoffs
    (distributed rate limiting, concurrency correctness, failure semantics)
- expert:
    Deep internals or correctness-critical specialist work
    (compiler/runtime/kernel internals, lock-free proofs, formal verification)
```

> *每个维度都有类似的锚点 + 校准指南，确保 LLM 标注一致性。*

Pass 2 — 价值评分：3 维度 12 子项

每个样本 **1 次 LLM 调用**，LLM 为每个子项独立打分（1–10）：

Complexity — 任务有多难?

- `instruction`：问题本身的挑战度
- `analytical_depth`：需要多少推理步骤
- `implementation`：代码实现的难度
- `overall`：LLM 综合判断

Reasoning — 推理质量

- `clarity`：推理链是否清晰可追踪
- `consistency`：推理是否逻辑自洽
- `self_correction`：是否有纠错意识（bool）
- `overall`：LLM 综合判断

Quality — 回答好不好? （**权重最高 40%**）

- `correctness`：代码/方案是否正确
- `code_quality`：命名、结构、最佳实践
- `explanation`：解释的清晰度和教学价值
- `completeness`：是否完整回答了指令
- `overall`：LLM 综合判断

每个维度的 `overall` 由 LLM 综合子项给出，最终 **value_score** 由三个 overall 加权：

```
value = 0.25×Complexity + 0.40×Quality
      + 0.20×Reasoning  + 0.15×Rarity
```

Pass 2 — Prompt 节选：Quality 评分锚点

```
## Dimension 2: Quality (1-10)
Rate the quality of the response. correctness is the most important sub-score.

Score anchors:
  1-2: Severely flawed - critical bugs, completely wrong approach
  3-4: Below average - partially correct, missing key aspects, poor code style
  5-6: Acceptable - mostly correct, adequate explanation, standard code quality
  7-8: Good - correct, well-structured, clear explanation, handles edge cases
  9-10: Excellent - production-quality code, insightful explanation, comprehensive

Calibration:
- A response with bugs in core logic is 3-4 at most for correctness,
  regardless of code style or explanation quality.
- overall should never exceed correctness by more than 2 points.
- Reserve 9-10 for code you would merge into production without changes.
- Expected distribution: ~15% at 1-3, ~50% at 4-6, ~30% at 7-8, ~5% at 9-10.
- CRITICAL: Most correct, working solutions score 5-6. Scoring 7+ requires
  NOTABLE qualities beyond correctness.
```

> *通过锚点 + 校准指南 + 期望分布，约束 LLM 避免"分数通胀"。*

Pass 2 — Prompt 节选：Agentic 复杂度量表

针对 Agent/Tool-use 场景，Prompt 中有专门的复杂度细分：

```
Agentic complexity granularity — use the full scale:

  3-4: Simple tool calls with linear flow (read file → apply fix → done)
  5:   Multi-step tool use with straightforward decision logic
  6:   Moderate orchestration — branching based on tool output, basic error recovery
  7:   Complex multi-tool coordination — iterative debugging loops,
       multiple files, conditional branching
  8:   Advanced orchestration — cross-file reasoning,
       hypothesis testing via tools, backtracking on failed approaches
  9-10: Novel agent strategies — creative tool composition,
        complex search through solution space
```

> *这套量表让 LLM 能区分"简单调工具"和"复杂 Agent 编排"的质量差异。*

Rarity — 基于标签分布的稀有度评分

Rarity 不由 LLM 打分，而是从 Pass 1 标签分布 **离线计算**，衡量样本的稀缺程度。

计算流程

1. **Per-tag IDF**：`log₂(N / (count+1))`
2. **多标签混合**：`0.6×max + 0.4×mean`
   避免单个稀有标签过度放大
3. **维度加权求和**（Concept 权重最高 2.0）
4. **组合稀有度**：标签组合的联合 IDF
5. **最终混合**：`0.7×tag_rarity + 0.3×combo_rarity`
6. **归一化到 1–10**

维度权重

维度	权重	维度	权重
Concept	2.0	Domain	1.5
Agentic	1.5	Language	1.0
Task	1.0	Constraint	1.0
Context	0.5	Difficulty	0.3
Intent	0.15		

Concept 和 Domain 对稀有度影响最大，Intent 影响最小。

Pass 2.5 — 多轮对话的标注策略

核心挑战：一条多轮对话包含多个 assistant turn，质量参差不齐。

策略：先切片标注，再聚合为对话级分数。

第一步：智能切片

每个 assistant turn 切为独立样本，携带完整前文上下文。

对话长度	策略
≤ 12 turns	全部逐 turn 标注
> 12 turns	**稀疏采样**：前 8 个全标，之后间隔指数增长（gap × 1.3，上限 8）

未标注的 turn 从最近的已标注前驱 **继承** 分数，标记 `inherited=true`，聚合时降权。

Pass 2.5 — 对话级聚合公式

第二步：turn 分数 → 对话分数（纯离线，无 LLM）

位置加权：后面的 turn 权重线性递增

```
weight(i, n) = 1.0 + i/(n-1)
// 第1轮=1.0 → 最后一轮=2.0
```

继承切片降权：inherited turn × 0.7

conv_value 计算：

```
base = 0.7 × weighted_avg(turn_values)
     + 0.3 × max(turn_values)
penalty = floor_penalty × 0.95^(neg_flags)
conv_value = clamp(base × penalty, 1, 10)
```

Quality floor penalty

quality_floor	penalty
< 3	× 0.5
< 5	× 0.8
⩾ 5	× 1.0

长对话 v2 公式（⩾8 turn / 有 tool）

```
signal = 0.70 × top5_mean
       + 0.20 × median
       + 0.10 × bottom3_mean
conv_v2 = 0.80 × signal + 0.20 × rarity
```

v2 不低于 v1，上限 v1 + 1.5

Pass 2.5 — 真实输出示例

以一条 44-turn SWE 轨迹为例（`coderforge-63`）：

对话级聚合结果

```
{
  "conversation_id": "coderforge-63",
  "turn_count": 44,
  "conv_value": 5.8,
  "conv_selection": 3.38,
  "peak_complexity": 7,
  "conv_rarity": 5.23,
  "tool_turn_ratio": 0.94,
  "unique_tools": [
    "execute_bash",
    "str_replace_editor", "finish"
  ],
  "thinking_mode": "slow"
}
```

单 turn 评分（turn 1/44）

```
{
  "complexity": {
    "instruction": 6, "analytical_depth": 6,
    "implementation": 6, "overall": 6
  },
  "quality": {
    "correctness": 8, "code_quality": 7,
    "explanation": 6, "completeness": 7,
    "overall": 7
  },
  "reasoning": {
    "clarity": 6, "consistency": 6,
    "self_correction": false, "overall": 6
  },
  "value_score": 4.1, "confidence": 0.85
}
```

> *turn 1 的 quality=7，但 conv_value=5.8 — 44 轮中有低质量 turn 拉低了整体。*

Value vs Selection — 两个最终分数

Value Score — 绝对质量

```
value = 0.25×Complexity + 0.40×Quality
      + 0.20×Reasoning  + 0.15×Rarity
```

- 衡量样本的 **绝对训练价值**
- 不考虑同类样本的分布
- 用于阈值筛选：`--value-min 6`
- quality < 4 时触发 × 0.7 惩罚

Selection Score — 相对优选

```
selection = 0.49 × intra_class_rank
          + 0.20 × pure_quality
          + 0.31 × rarity
```

- 衡量样本在 **同类中的相对排名**
- `intra_class_rank`：按标签分组，组内 percentile + 贝叶斯平滑
- 用于多样性筛选：`--selection-min 6`
- 高 value 但同类多 → selection 较低
- 中等 value 但标签稀有 → selection 较高

> *Value 回答"这条数据好不好"，Selection 回答"这条数据该不该选"。*

Pass 3 & 4 — 语义去重 & 筛选导出

Pass 3 — 语义聚类

针对长轨迹中的冗余片段：

- 双语 role-aware 文本渲染 → 向量化
- SemHash 确定性投影 + ANN 余弦精排
- 固定任务前缀（pinned prefix）保留上下文
- 按 **信噪比（SNR）** 选代表性窗口

Pass 4 — 多条件筛选

```
# 样本级
sft-label filter --value-min 6 \
  --difficulty advanced,expert

# 对话级
sft-label filter --conv-value-min 7 \
  --turn-count-min 3 --format training
```

支持：value / selection / tags / difficulty / thinking_mode / conv_value / turn_count / peak_complexity

数据格式 & 运行模式

支持的格式

ShareGPT — 业界最通用

```
{"conversations": [
  {"from": "human", "value": "..."},
  {"from": "gpt", "value": "..."}
]}
```

Pangu — 支持 COT 和伪多轮

```
{"data": [
  {"role": "user", "content": "..."},
  {"role": "assistant", "content": "..."}
]}
```

三种运行模式

Mode 1 — 单文件

一个 JSON → 一个 run_dir

Mode 2 — 目录批量

JSON 目录 → 按文件并发处理

Mode 3 — Inline JSONL

标签嵌入原始数据行，镜像目录树

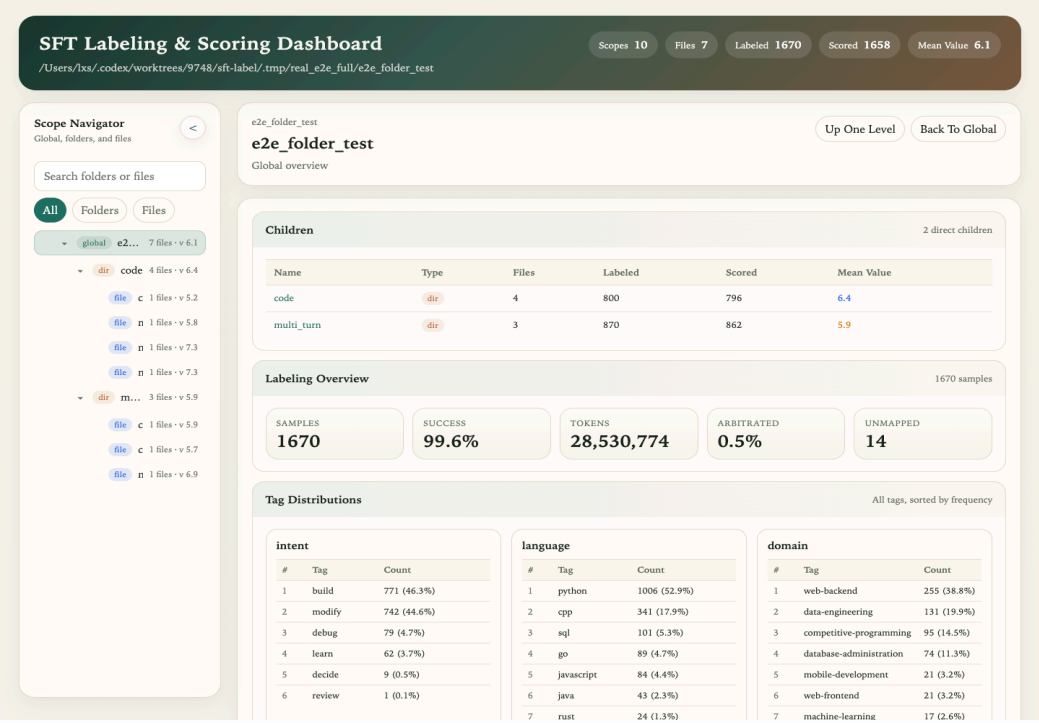
适合迭代式 curation

```
uv run sft-label start  # 交互式自动选择
```

Dashboard — 交互式质量报告

- 自动生成交互式 HTML 报告
- 标注分布 & 评分分布可视化
- 支持静态托管，稳定 URL
- 跨 run 聚合统计

```
sft-label dashboard-service init \
  --web-root ~/sft-dashboard
sft-label dashboard-service start
```



技术架构亮点

特性	实现
异步并发	asyncio + httpx，批量 LLM 调用
LLM 后端	LiteLLM 统一接口，可切换任意模型
断点续跑	Checkpoint 机制，中断后自动恢复
速率控制	内置 rate limiting + timeout 递增
COT 处理	评分时保留思维链，切片时移除避免误判
一致性校验	Tag Pool 白名单 + 跨维度规则验证
置信度追踪	每个样本附带评估置信度 0.0–1.0

竞品全景：SFT 数据质量赛道

工具	方法	多维评分	多轮支持	成熟度
sft-label	LLM-as-Judge prompt	✅ 9维+12子项	✅ 切片+聚合	CLI 工具
Distilabel (Argilla/HF)	LLM-as-Judge 框架	部分	流水线级	活跃框架
Deita (HKUST-NLP)	训练好的神经评分器	Quality+Complexity	❌	研究代码
Alpagasus	GPT/Claude 单分评分	❌ 单一总分	❌	研究原型
InsTag (Alibaba)	细粒度语义标签	标签非评分	❌	研究代码
Cherry LLM	自评 IFD 难度	❌ 单一维度	❌	研究代码

竞品详解（1/2）

Distilabel (Argilla / HF)

最接近的框架级方案。内置 QualityScorer、ComplexityScorer、UltraFeedback 等评分组件。

- 定位：**合成数据 + 评分** 全流水线
- 差异：评分组件分散，无对话级聚合
- 更适合"生成后过滤"场景

Alpagasus (2023)

LLM-as-Judge 筛选 SFT 数据的开山之作。ChatGPT 打 1-5 分，阈值过滤（52K → 9K）。

- 差异：单一总分，无多维 rubric
- 研究原型，未持续维护

Deita (HKUST-NLP, 2024)

最强的研究竞品。训练专用神经评分器，评 Quality + Complexity。6K 筛选样本可匹配 100K+ 未筛选效果。

- 差异：评分器固定，无法通过 prompt 自定义评分标准，不支持多轮

InsTag (Alibaba OFA-Sys)

标签体系方向。6,600+ 细粒度语义标签，通过标签分布衡量多样性和复杂度。

- 差异：只做标签不做评分
- 可作为互补工具

竞品详解（2/2）与差异化定位

Cherry LLM / SuperFiltering

自评估方向。用目标模型自身或小模型代理计算 IFD 难度分数。

- 优势：零 API 成本
- 劣势：只有难度维度，无质量判断

常被误认为竞品的工具

工具	实际定位
DeepEval	LLM 应用测试框架
Data-Juicer	数据 ETL 平台
NeMo Curator	GPU 预训练数据清洗
AlpacaEval	模型排行榜

sft-label 的差异化

现有工具各做一块：

- Deita / Alpagasus：评分但无多轮
- InsTag：标签但无评分
- Distilabel：框架但非专用 CLI

sft-label 做完整闭环：

- 9 维分类 + 12 子项评分
- 多轮切片 + 对话级聚合
- Value + Selection 双分数体系
- 语义去重 + 多条件筛选
- 轻量 CLI + LiteLLM 可换模型

总结

用 LLM 评估 LLM 的训练数据

9 维分类 · 225+ 标签 — 3 维评分 · 12 子项 — Value + Selection 双分数

多轮切片 · 对话聚合 — 语义去重 · 灵活筛选

```
uv run sft-label start
```

Q & A

有问题随时提问